

Scalable Services Assignment

Microservices-Based Application Development

Assignment: Problem Statement 1 – Banking

GROUP - 07

Team Members & Roles

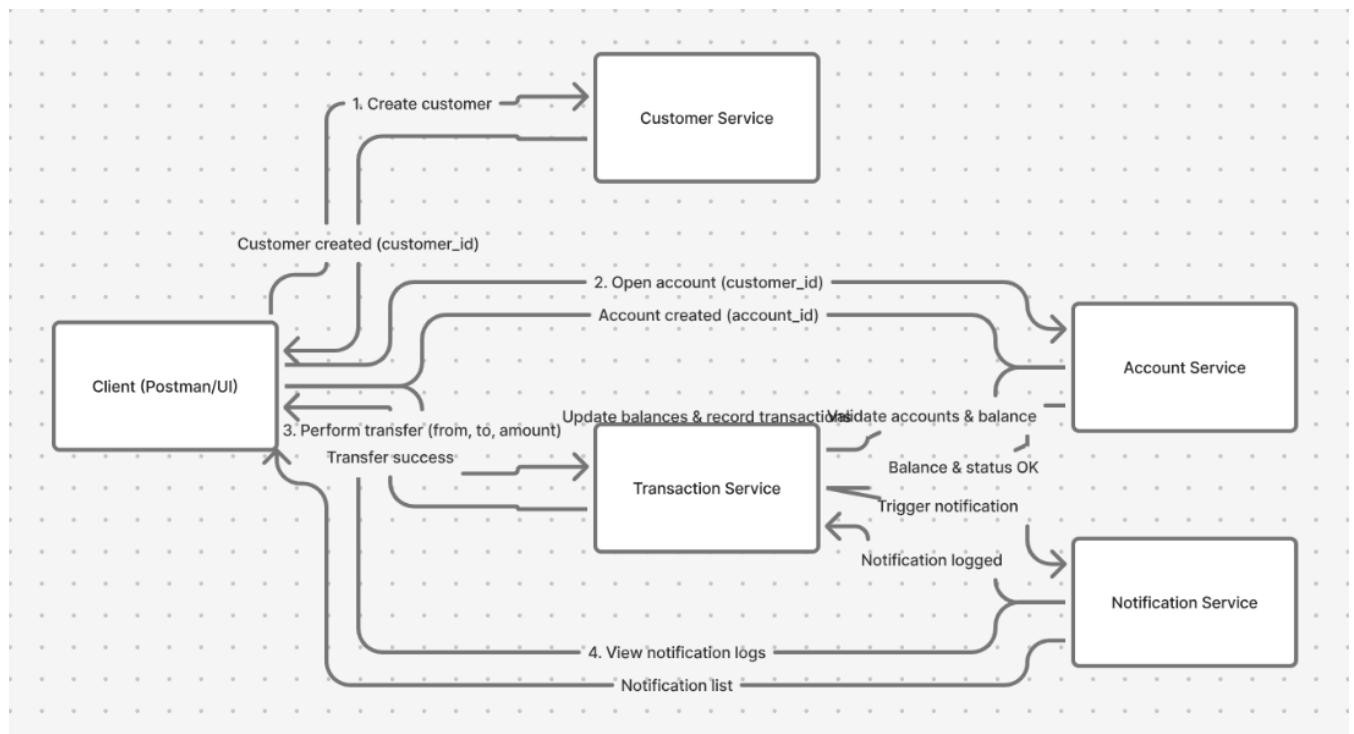
- Himanshu S Gautam (2024TM93048) – Lead Developer (Transaction Service, Idempotency, Business Rules)
- Mallidi Akhil Reddy (2024TM93056) – Customer & Account Services, Database Design
- Kajal (2024TM93044) – DevOps (Docker, Minikube), Notification Service, Monitoring & Logs
 - Inacio Fernandes (2024TM93050) – Documentation & Testing

Summary

We implemented a banking system as four independent microservices (Customer, Account, Transaction, Notification) using Python, FastAPI, and PostgreSQL, containerized with Docker and deployed on a local Minikube Kubernetes cluster.

Objectives

- Wrap the provided **banking transactions database** into a **microservices-based application** (minimum 4 services).
- Apply the **Database-per-Service** design pattern: each service owns its schema; no shared tables.
- Implement **loosely coupled communication** between services using REST (synchronous) and clear API contracts.
- Containerize each service in its own **Docker image**.
- Deploy the full system on **Minikube**, with basic **monitoring and logging**.



Technology Stack

- **Language & Framework:**
 - Python 3.x
 - FastAPI

- Uvicorn (ASGI server)
- **Database:**
 - PostgreSQL
 - Data seeded from:
 - `customers.csv`
 - `accounts.csv`
 - `transactions.csv`
- **ORM / DB Access:**
 - SQLAlchemy / async DB driver (as per implementation)
- **Containerization & Orchestration:**
 - Docker, Docker Compose
 - Kubernetes (Minikube)
- **Config & Secrets:**
 - Environment variables for local development
 - ConfigMaps & Secrets for Kubernetes (DB credentials, SMTP config)
- **Monitoring & Logging:**
 - FastAPI/Uvicorn logging
 - Optional Prometheus-style `/metrics` endpoint
 - Structured JSON logs with correlation IDs

System Architecture

Microservices Overview

We implemented the following microservices, each in its own repository and container:

1. Customer Service

- a. Manages customer profiles and KYC status.
- b. Owns the `customers` table.

2. Account Service

- a. Manages bank accounts, balances, and account status.
- b. Owns the `accounts` table.

3. Transaction Service

- a. Handles deposits, withdrawals, and transfers.
- b. Owns the `transactions` table.
- c. Enforces business rules and idempotency for `/transfer`.

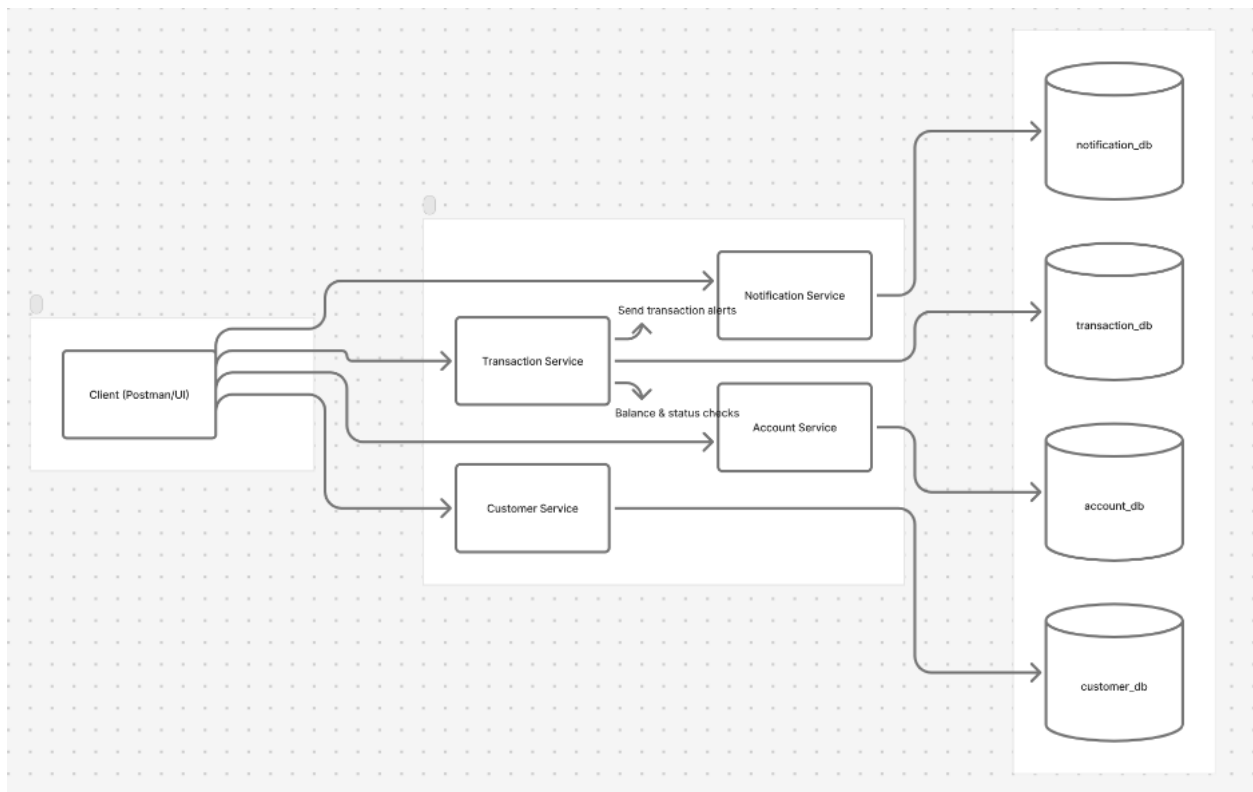
4. Notification Service

- a. Sends/simulates email/SMS notifications for high-value transactions and account status changes.
- b. Owns `notifications_log`.

Each service exposes a REST API with automatic Swagger/OpenAPI documentation at `/docs`.

High-Level Architecture (Textual View)

- Clients (Postman / curl / browser) call the services via HTTP.
- **Transaction Service** calls:
 - **Account Service** to check balances and account status.
 - **Notification Service** to push alerts for selected events.
- Each service connects only to its own PostgreSQL schema/database.
- All containers run in a Minikube cluster namespace (for Kubernetes deployment).



Database Design

We started from the given CSV datasets (`customers.csv`, `accounts.csv`, `transactions.csv`) and created corresponding tables in PostgreSQL. We follow a **Database-per-Service** logical pattern – each service owns and primarily uses the table(s) relevant to its domain.

Customers Table (Customer Service)

Table name: `customers`

Key columns (aligned with `customers.csv`):

- `customer_id` – INT, Primary Key
- `name` – VARCHAR(100), not null
- `email` – VARCHAR(255), not null, unique
- `phone` – BIGINT, not null
- `kyc_status` – VARCHAR(20), not null (e.g., PENDING, VERIFIED, REJECTED)
- `created_at` – TIMESTAMP, not null

Purpose

- Source of truth for customer identities and KYC status.
- Used by Customer Service for all customer CRUD operations.

Example DDL

```
CREATE TABLE customers (  
    customer_id INT PRIMARY KEY,  
    name        VARCHAR(100) NOT NULL,  
    email       VARCHAR(255) NOT NULL UNIQUE,  
    phone       BIGINT NOT NULL,  
    kyc_status  VARCHAR(20) NOT NULL,  
    created_at  TIMESTAMP NOT NULL  
);
```

Accounts Table (Account Service)

Table name: accounts

Key columns (aligned with accounts.csv):

- account_id – INT, Primary Key
- customer_id – INT, not null (links to customer)
- account_number – BIGINT, unique, not null
- account_type – VARCHAR(20), not null (e.g., SAVINGS, CURRENT)
- balance – NUMERIC(15,2), not null
- currency – VARCHAR(10), not null (e.g., INR)
- status – VARCHAR(20), not null (ACTIVE, FROZEN, CLOSED)
- created_at – TIMESTAMP, not null

Relationships

- One customer can have many accounts.
- At the DB level we may use a foreign key:

```
CONSTRAINT fk_accounts_customers  
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
```

Example DDL

```
CREATE TABLE accounts (  
    account_id      INT PRIMARY KEY,  
    customer_id     INT NOT NULL,  
    account_number  BIGINT NOT NULL UNIQUE,  
    account_type    VARCHAR(20) NOT NULL,  
    balance         NUMERIC(15,2) NOT NULL,  
    currency        VARCHAR(10) NOT NULL,  
    status          VARCHAR(20) NOT NULL,  
    created_at      TIMESTAMP NOT NULL,  
    CONSTRAINT fk_accounts_customers  
        FOREIGN KEY (customer_id) REFERENCES customers(customer_id)  
);
```

Transactions Table (Transaction Service)

Table name: transactions

Key columns (aligned with transactions.csv):

- txn_id – INT, Primary Key
- account_id – INT, not null
- amount – NUMERIC(15,2), not null
- txn_type – VARCHAR(10), not null (DEBIT, CREDIT)
- counterparty – VARCHAR(255), nullable
- reference – VARCHAR(255), nullable
- created_at – TIMESTAMP, not null

Relationships

- Each transaction belongs to exactly one account.
- Foreign key:

```
CONSTRAINT fk_transactions_accounts  
    FOREIGN KEY (account_id) REFERENCES accounts(account_id)
```

Example DDL

```
CREATE TABLE transactions (  
    txn_id          INT PRIMARY KEY,  
    account_id     INT NOT NULL,  
    amount         NUMERIC(15,2) NOT NULL,  
    txn_type       VARCHAR(10) NOT NULL,  
    counterparty   VARCHAR(255),  
    reference      VARCHAR(255),  
    created_at     TIMESTAMP NOT NULL,  
    CONSTRAINT fk_transactions_accounts  
        FOREIGN KEY (account_id) REFERENCES accounts(account_id)  
);
```

Notification Log Table (Notification Service)

Table name: notifications_log

Columns:

- id – SERIAL, Primary Key
- type – VARCHAR(50) (e.g., HIGH_VALUE_TXN, STATUS_CHANGE)
- target – VARCHAR(255) (email/phone)
- payload – TEXT (JSON or message body)
- status – VARCHAR(20) (e.g., SENT, FAILED)
- created_at – TIMESTAMP, not null

Example DDL

```
CREATE TABLE notifications_log (  
    id          SERIAL PRIMARY KEY,  
    type        VARCHAR(50) NOT NULL,  
    target      VARCHAR(255) NOT NULL,  
    payload     TEXT NOT NULL,  
    status      VARCHAR(20) NOT NULL,  
    created_at  TIMESTAMP NOT NULL DEFAULT NOW()  
);
```

Data Seeding from CSVs

- The initial data for **customers**, **accounts**, and **transactions** is loaded from:
 - customers.csv → customers table
 - accounts.csv → accounts table
 - transactions.csv → transactions table

This provides realistic sample data (~60 customers, ~90 accounts, ~300 transactions) for testing.

Microservices & API Design

Below are the core endpoints for each service (typical, you can adjust to match your code).

Customer Service

Base URL (local example): <http://localhost:8001>

Key endpoints:

- POST /customers
 - Create a new customer.
 - Request body: name, email, phone, kyc_status.
- GET /customers/{customer_id}
 - Return customer details.
- PUT /customers/{customer_id}
 - Update customer data (e.g., KYC status).
- GET /customers
 - List customers (with optional pagination).

Swagger UI: /docs

Account Service

Base URL (local example): <http://localhost:8002>

Key endpoints:

- POST /accounts
 - Open a new account for a customer.
- POST /accounts/{account_id}/close
 - Close an account (status → CLOSED).
- POST /accounts/{account_id}/freeze
 - Freeze an account (status → FROZEN).
- GET /accounts/{account_id}
 - Get account details.
- GET /accounts/{account_id}/balance

- Return current balance and status.

Transaction Service

Base URL (local example): <http://localhost:8003>

Key endpoints:

- POST /transactions/deposit
 - Deposit to an account.
 - Request: account_id, amount, reference.
- POST /transactions/withdraw
 - Withdrawal from an account (with overdraft checks).
- POST /transactions/transfer
 - Transfer between two accounts.
 - Request: from_account_id, to_account_id, amount, reference.
 - Header: Idempotency-Key: <UUID>
 - Enforces idempotency by storing the key and preventing duplicate transfers.
- GET /transactions/account/{account_id}
 - Return list/statement of transactions for given account.

Business Rules Implemented

- No overdraft allowed for standard accounts.
- Transfers blocked if source account is FROZEN or CLOSED.
- Optional: daily transfer limit per account (e.g., 2,00,000).
- Idempotent /transfer using Idempotency-Key.

Notification Service

Base URL (local example): <http://localhost:8004>

Key endpoints:

- POST /notifications/transaction
 - Called by Transaction Service for high-value transactions.

- Logs a notification and simulates email/SMS sending.
- POST /notifications/account-status
 - Called by Account Service when account status changes (e.g., freeze/close).
- GET /notifications/logs
 - Returns recent entries from notifications_log for verification.

Inter-Service Communication

- Transaction Service → Account Service:
 - HTTP REST for balance checks, account status, and basic validation.
- Transaction Service → Notification Service:
 - HTTP REST to send high-value transaction alerts.
- Account Service → Notification Service:
 - HTTP REST to send account status change alerts.

Timeouts and basic error handling are used to ensure the system fails gracefully when a downstream service is unavailable.

Containerization (Docker)

Each microservice has:

- Its own **Dockerfile**:
 - Uses python:3.12-slim (or similar) as base image.
 - Installs dependencies from requirements.txt.
 - Copies source code.
 - Runs uvicorn main:app --host 0.0.0.0 --port <port>.
- Environment variables for DB connectivity:
 - DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD.

A docker-compose.yml (if used) can:

- Start all four services.
- Start one PostgreSQL container per logical DB (or one DB with multiple schemas).

- Configure networks so services can talk to their respective DBs and each other by service name.

Customer service docker:

```
customer-service -- -zsh -- 145x24

malakhil@WKMT31CF475 customer-service %
malakhil@WKMT31CF475 customer-service %
malakhil@WKMT31CF475 customer-service % docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
930e7d58ffbe   banking-services-customer-service   "docker-entrypoint.s..." 2 hours ago    Up 4 seconds   0.0.0.0:5001->5001/t
cp, [::]:5001->5001/tcp
b6e6bccc212    postgres:16                         "docker-entrypoint.s..." 9 hours ago    Up 10 seconds (healthy)   0.0.0.0:5434->5432/t
cp, [::]:5434->5432/tcp
b916bfd2fd56   postgres:16                         "docker-entrypoint.s..." 9 hours ago    Up 10 seconds (healthy)   0.0.0.0:5432->5432/t
cp, [::]:5432->5432/tcp
cb0995e675de   postgres:16                         "docker-entrypoint.s..." 9 hours ago    Up 10 seconds (healthy)   0.0.0.0:5433->5432/t
cp, [::]:5433->5432/tcp
6c2f140234f1   rabbitmq:3-management              "docker-entrypoint.s..." 9 hours ago    Up 10 seconds (health: starting) 0.0.0.0:5672->5672/t
cp, [::]:5672->5672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp
2d80df723967   mongo:6.0                          "docker-entrypoint.s..." 9 hours ago    Up 10 seconds (healthy)   0.0.0.0:27017->27017
/tcp, [::]:27017->27017/tcp
malakhil@WKMT31CF475 customer-service %
```

Banking services docker:

```
banking-services -- -zsh -- 145x31

=> CACHED [4/6] RUN npm install 0.0s
=> CACHED [5/6] RUN npm ci --omit=dev 0.0s
=> CACHED [6/6] COPY . . 0.0s
=> exporting to image 0.0s
=> => exporting layers 0.0s
=> => exporting manifest sha256:72cdcc4e2f6fd0590b67ea6b1e779a86d70aaf8a780abcbad3f322f6fd99df9b 0.0s
=> => exporting config sha256:fd76d8fd42c1ae7659fcc5f7237cbdae5c2b71b02c7e8468a773b1597d858e0f 0.0s
=> => exporting attestation manifest sha256:1681a3b3c6848d4d65ccbe0c99581b74b668b0be0017a83d809d61b73b5af990 0.0s
=> => exporting manifest list sha256:0827b4ba437ece696a9a618dd0ece9a1c1867df1f4a89b1c8189aadda56c61f2 0.0s
=> => naming to docker.io/library/notification-service:1.0 0.0s
=> => unpacking to docker.io/library/notification-service:1.0 0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/693sp2ol2cf7yhnyfqu7nhzz
malakhil@WKMT31CF475 banking-services % docker images
REPOSITORY          TAG          IMAGE ID      CREATED        SIZE
banking-services-transaction-service   latest      a680a276bf22 2 hours ago    292MB
transaction-service                    1.0         c40a4b530775 2 hours ago    292MB
banking-services-account-seed          latest      2f42ebef042f 2 hours ago    335MB
banking-services-account-service       latest      0c64aff52a99 2 hours ago    335MB
account-service                        1.0         f638c46ce6e5 2 hours ago    335MB
banking-services-customer-service      latest      17e8c0600b6e 12 hours ago   275MB
banking-services-customer-seed         latest      bbbfa32dfc31 12 hours ago   275MB
customer-service                       1.0         7e2c078b4eb3 12 hours ago   275MB
notification-service                    1.0         0827b4ba437e 26 hours ago   318MB
banking-services-notification-service   latest      e36cbd5c5cd4 26 hours ago   318MB
postgres                               16          21f6013073bc 5 days ago     657MB
mongo                                   6.0         0a83b2f824b2 5 weeks ago    999MB
gcr.io/k8s-minikube/kicbase            <none>      7171c97a5162 2 months ago   1.73GB
gcr.io/k8s-minikube/kicbase            v0.0.48    18b0fda1fa3d 2 months ago   1.73GB
rabbitmq                                3-management 3b65f271d3e6 13 months ago 405MB
malakhil@WKMT31CF475 banking-services %
```

```

malakhil@WKMT31CF475 banking-services % kubectl get svc
NAME                                TYPE        CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
account-db                          ClusterIP    None             <none>            5432/TCP         8h
account-service                     NodePort     10.109.99.187    <none>            8081:31502/TCP   8h
customer-service                     NodePort     10.108.110.238   <none>            5001:31501/TCP   8h
kubernetes                           ClusterIP    10.96.0.1        <none>            443/TCP          9h
mongo                               ClusterIP    None             <none>            27017/TCP        8h
notification-service                 NodePort     10.98.71.143     <none>            5004:31504/TCP   8h
postgres-notification                ClusterIP    None             <none>            5432/TCP         8h
postgres-transaction                 ClusterIP    None             <none>            5432/TCP         8h
rabbitmq                             ClusterIP    10.103.229.156   <none>            5672/TCP,15672/TCP 8m3s
transaction-service                  NodePort     10.103.83.77     <none>            3000:31503/TCP   8h
malakhil@WKMT31CF475 banking-services %

```

Notification docker:

The screenshot shows the DBeaver 25.2.4 interface with the 'notifications' table selected. The table contains 20 rows of transaction events, including errors and deposits/withdrawals. The 'event_type' column contains JSON payloads with details like timestamp, customer_id, account_id, and amount.

event_type	account_id
transaction.error	123
transaction.error	123
transaction.error	123
transaction.error	123
transaction.error	123
transaction.error	123
transaction.error	123
transaction.error	123
transaction.error	123
transaction.error	123
transaction.error	123
transaction.error	123
transaction.error	123
transaction.error	123
transaction.error	123
transaction.error	123
transaction.error	123
transaction.error	123
transaction.error	123
transaction.error	123
transaction.error	123

Customer Docker:

BITS Pilani | Scalable Services | PS 1 Banking | Group - 07

Compass

My Queries

Data Modeling

CONNECTIONS (1)

Search connections

localhost:27017

- admin
- config
- customerdb
 - customers
- local

localhost:27017 > customerdb > customers

Documents 60 Aggregations Schema Indexes 2 Validation

Type a query: { field: 'value' } or [Generate query](#)

EXPLAIN RESET FIND Options

ADD DATA EXPORT DATA UPDATE DELETE

25 1 - 25 of 60

```
{
  "_id": ObjectId("6910e255a3f27a521c40b95f"),
  "name": "Om Patel",
  "email": "om.patel26@banka1l.in",
  "phone": "9211755002",
  "kyc_status": "REJECTED",
  "created_at": 2022-02-24T20:26:03.000+00:00,
  "_v": 0,
  "createdat": 2025-11-09T18:49:57.949+00:00,
  "updatedat": 2025-11-09T18:49:57.949+00:00
}
```

```
{
  "_id": ObjectId("6910e255a3f27a521c40b960"),
  "name": "Reyansh Kapoor",
  "email": "Reyansh.kapoor430@gmail.com",
  "phone": "9428714227",
  "kyc_status": "PENDING",
  "created_at": 2022-07-09T13:49:57.000+00:00,
  "_v": 0,
  "createdat": 2025-11-09T18:49:57.949+00:00,
  "updatedat": 2025-11-09T18:49:57.949+00:00
}
```

```
{
  "_id": ObjectId("6910e255a3f27a521c40b961"),
  "name": "Vihan Chatterjee",
  "email": "Vihan.chatterjee88@banka1l.in",
  "phone": "9687593566",
  "kyc_status": "VERIFIED",
  "created_at": 2025-03-28T07:00:03.000+00:00,
  "_v": 0,
  "createdat": 2025-11-09T18:49:57.949+00:00,
  "updatedat": 2025-11-09T18:49:57.949+00:00
}
```

```
{
  "_id": ObjectId("6910e255a3f27a521c40b962"),
  "name": "Kabir Gupta",
  "email": "Kabir.gupta47@gmail.com",
  "phone": "979834613",
  "kyc_status": "REJECTED",
  "created_at": 2023-07-01T23:35:39.000+00:00,
  "updatedat": 2025-11-09T18:49:57.949+00:00
}
```

Postman: Accounts service:

account-service / /accounts/id

GET http://localhost:8081/accounts/121

Params Authorization Headers (6) Body Scripts Settings Cookies

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers (4) Test Results 200 OK · 7 ms · 336 B Save Response

JSON Preview Visualize

```
1 {
2   "account_id": 121,
3   "customer_id": "6910e2a68509c3d8f4c950f0",
4   "account_number": "723121005203",
5   "account_type": "SALARY",
6   "balance": 49608.8,
7   "currency": "INR",
8   "status": "ACTIVE",
9   "created_at": "2025-11-09T18:53:55.145244"
10 }
```

account-service / /accounts

GET http://localhost:8081/accounts

Params Authorization Headers (6) Body Scripts Settings Cookies

Query Params

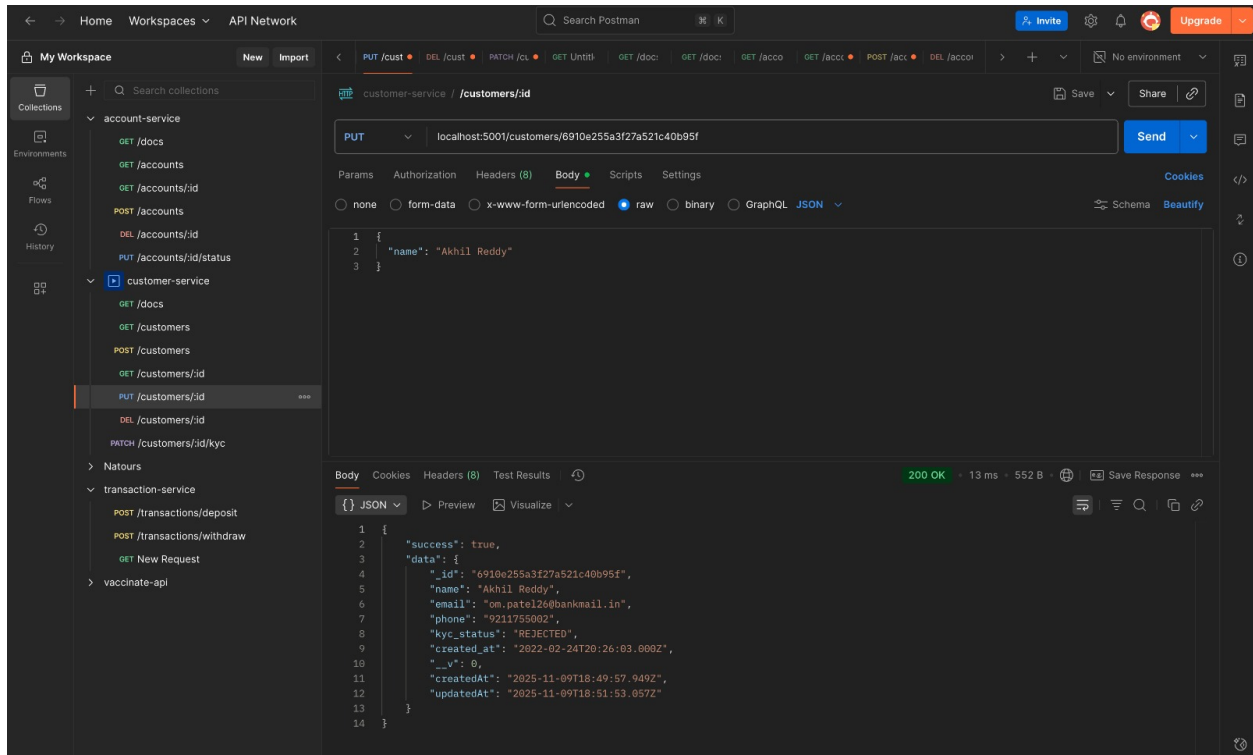
Key	Value	Description
Key	Value	Description

Body Cookies Headers (4) Test Results 200 OK · 25 ms · 16.68 KB Save Response

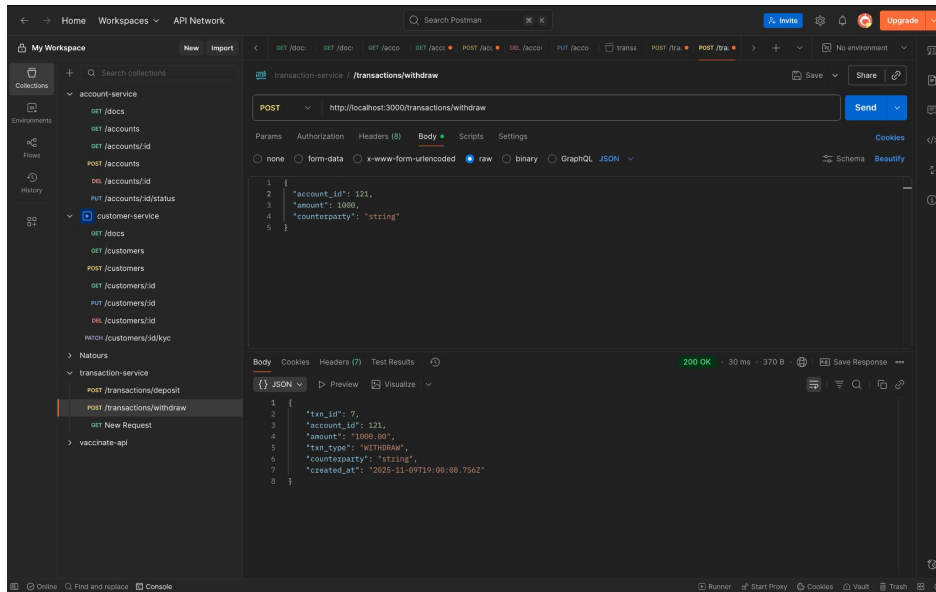
JSON Preview Visualize

```
1 [
2   {
3     "account_id": 3,
4     "customer_id": "2",
5     "account_number": "526468425099",
6     "account_type": "SALARY",
7     "balance": 319830.41,
8     "currency": "INR",
9     "status": "ACTIVE",
10    "created_at": "2023-09-11T07:15:11"
11  },
12  {
13    "account_id": 4,
14    "customer_id": "3",
15    "account_number": "81332683987",
16    "account_type": "SAVINGS",
17  }
18 ]
```

Postman: Customer:



Postman: Transactions:



BITS Pilani | Scalable Services | PS 1 Banking | Group - 07

The screenshot shows the Postman application interface. On the left, the 'My Workspace' sidebar lists collections for 'account-service' and 'customer-service'. The 'customer-service' collection is expanded, showing endpoints like '/docs', '/customers', and '/transactions'. The main panel displays a 'New Request' for the endpoint 'http://localhost:3000/transactions/statement/121' using the GET method. The 'Query Params' table is empty. The 'Body' tab shows a JSON response with a status of '200 OK', a response time of '22 ms', and a size of '508 B'. The response body is a JSON array of two transaction objects.

```
1 [
2   {
3     "txn_id": 7,
4     "account_id": 121,
5     "amount": "1000.00",
6     "txn_type": "WITHDRAW",
7     "counterparty": "string",
8     "created_at": "2025-11-09T19:00:08.756Z"
9   },
10  {
11    "txn_id": 6,
12    "account_id": 121,
13    "amount": "50000.00",
14    "txn_type": "DEPOSIT",
15    "counterparty": "string",
16    "created_at": "2025-11-09T18:58:39.043Z"
17  }
18 ]
```

Postman: Customer service:

The screenshot shows the Postman application interface. On the left, the 'My Workspace' sidebar lists collections for 'account-service' and 'customer-service'. The 'customer-service' collection is expanded, showing endpoints like '/docs', '/customers', and '/health'. The main panel displays a 'New Request' for the endpoint 'localhost:5001/health' using the GET method. The 'Query Params' table is empty. The 'Body' tab shows a JSON response with a status of '200 OK', a response time of '6 ms', and a size of '311 B'. The response body is a JSON object indicating the service is 'UP'.

```
1 {
2   "status": "UP",
3   "service": "customer-service"
4 }
```

Kubernetes Deployment (Minikube)

Setup

- Start Minikube:
`minikube start`
- Create namespace (optional):
`kubectl create namespace banking-ms`

Kubernetes Manifests

For each service (Customer, Account, Transaction, Notification):

- **Deployment:**
 - References the Docker image:
`image: <dockerhub-username>/<service-name>:<tag>`
 - Environment variables from ConfigMap/Secret.
 - `livenessProbe` and `readinessProbe` on `/health` or root path.
- **Service:**
 - ClusterIP for internal communication.
 - NodePort or Ingress for external access (especially Transaction Service or an API gateway).

For each PostgreSQL instance:

- Deployment or StatefulSet for the DB.
- PersistentVolumeClaim to store data.
- Service (ClusterIP) for access from the corresponding microservice.

End-to-End Flow in Minikube

Typical demo steps:

1. `kubectl apply -f k8s/postgres/`
2. `kubectl apply -f k8s/services/`
3. `kubectl get pods -n banking-ms` – verify all pods are running.
4. `kubectl get svc -n banking-ms` – get service endpoints.
5. `minikube service transaction-service -n banking-ms --url` – get URL.
6. Use that URL in Postman / curl to:

- a. Create a customer
- b. Open an account
- c. Perform a transfer
- d. Call Notification logs endpoint to verify alert entries.

Monitoring & Logging

Logs

- Structured JSON logs where possible, including:
 - Timestamp
 - Service name
 - Log level
 - Endpoint, HTTP method, status code
 - `correlation_id` / `trace_id` (propagated between services)
- Sensitive fields (email, phone) are partially masked in logs.

Metrics

Example metrics:

- Technical:
 - Request count per endpoint
 - Error count and rate
 - Latency histograms (p50/p90/p99)
- Business:
 - `transactions_total{type="deposit|withdraw|transfer"}`
 - `failed_transfers_total{reason="overdraft|frozen|limit_exceeded"}`
 - `balance_check_latency_ms`

Optional: Prometheus/Grafana dashboard visualizing RPS, error rate, and latency.

Testing & Example Scenarios

Functional Tests

- **Create customer** → assert status 201 and record exists.
- **Open account** → assert account linked to correct customer and balance = initial.
- **Deposit** → balance increases correctly and transaction recorded.
- **Withdraw** → allowed only if sufficient balance; otherwise returns clear error.
- **Transfer:**
 - From Account A to Account B.
 - Balance decreases in A and increases in B.
 - Two transaction records were created (debit and credit).
 - A high-value transfer triggers Notification Service.
- **Idempotent transfer:**
 - Same request with same Idempotency-Key returns same result without double charging.

Non-Functional Checks

- Behavior when Account Service is down (Transaction Service handles failure gracefully).
- Behavior when Notification Service is down (transfer may succeed, but alert may fail – logged appropriately).
- Basic performance sanity (e.g., multiple concurrent requests).

DEMO Link: [Click here for demo link](#)

GitHub Repositories

- Customer Service: <https://github.com/itzzmeakhi/customer-service>
- Account Service: <https://github.com/itzzmeakhi/banking-services>
- Transaction Service:
https://github.com/himanshugy2j/Banking_Transaction_Microservices

- Notification Service:
https://github.com/himanshugy2j/Banking_Notification_Microservice

Limitations & Future Enhancements

Current Limitations

- Simple authentication or no authentication (depending on implementation).
- Synchronous RPC-style REST; no message queue for events.
- Limited validation and error messages in some endpoints.

Future Enhancements

- Add JWT/OAuth2-based security and role-based access control.
- Introduce message broker (Kafka/RabbitMQ) and event-driven notifications.
- Implement fully distributed tracing with OpenTelemetry and a tracing backend.
- Add a simple React/Angular frontend for end-user interaction.